

Enabling Floating Point Virtualization With Tiny Numbers

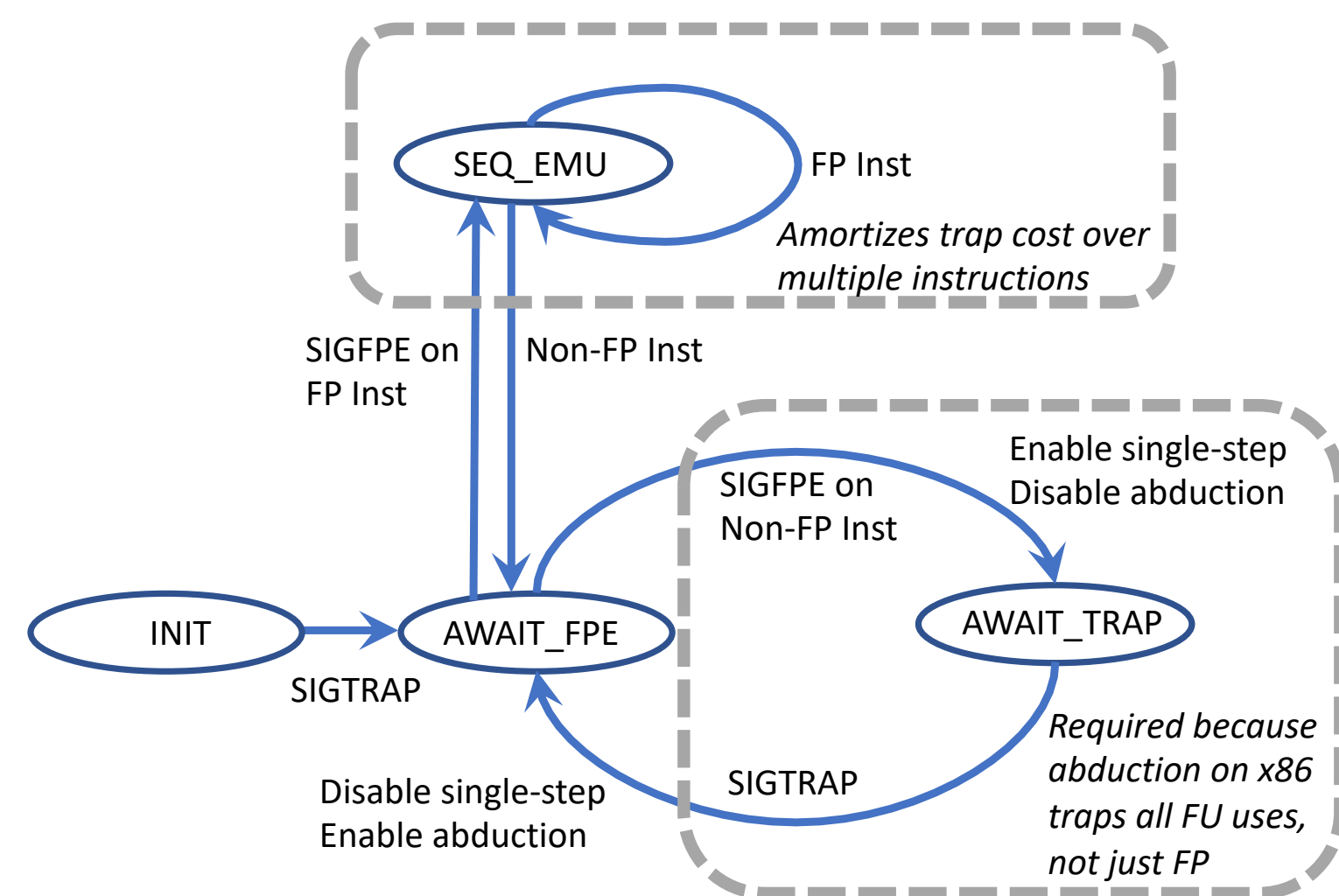
Kevin Hayes Peter Dinda

How do we make porting *existing* programs to low-precision floating point hardware easy?

- Increased demand for low-precision hardware (often driven by AI/ML workloads)
- Can low-precision hardware be utilized for arbitrary workloads?
- Problem:** correctly porting a program to low-precision floating point is *hard*
- Solution:** Floating Point Virtualization!

Standard Floating Point Virtualization “Leaks” High Precision

- FPVM [1] virtualizes **higher** precision floating point (GNU MPFR, Posits, etc.) on **unmodified binaries** via userspace trap-and-emulate virtualization.



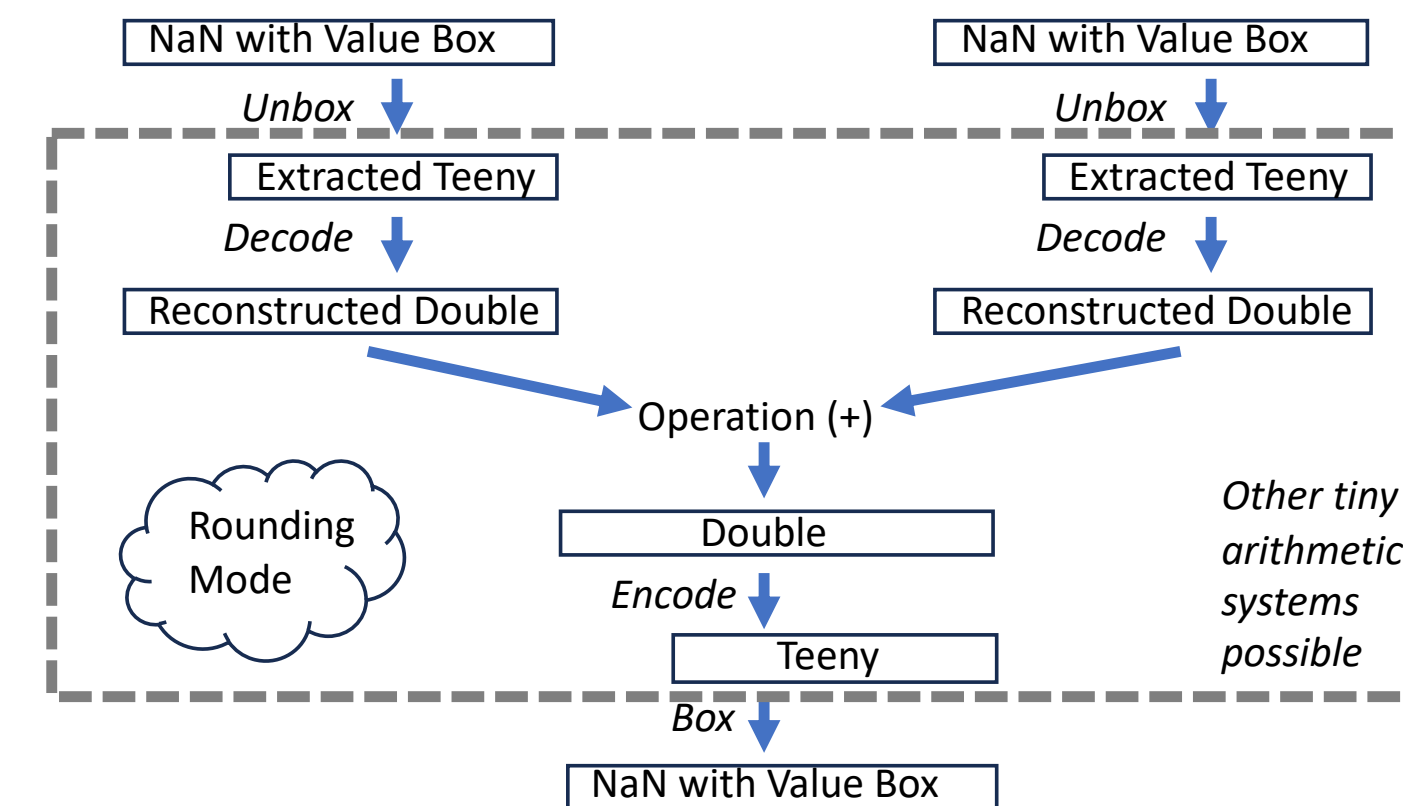
- Computation precisely representable by 64-bit “doubles” are **invisible** to FPVM (e.g. *they do not trap*)

Lazy FPU Abduction Fixes The Leaks

- Our technique [2] makes *every* floating point operation visible to FPVM.
- Leverages existing but currently unused hardware features designed to allow lazily switching floating-point registers on a context switch
- We expose this functionality to userspace through a small (<1000 LoC) Linux kernel module which is **fully insertable/removable**.

“Teeny” Arithmetic

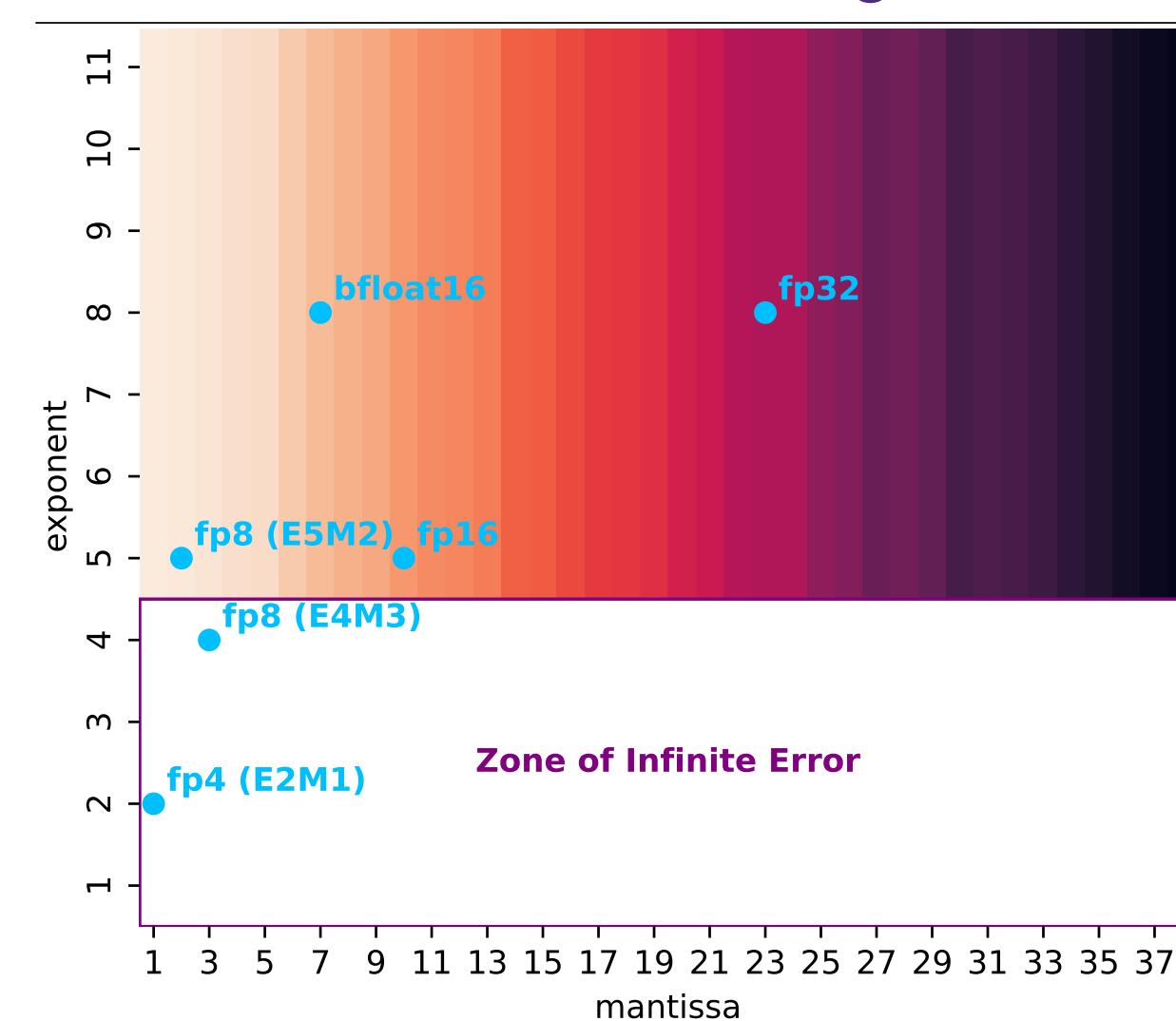
- Encode a low-precision value *within the payload of a double precision NaN*.
- For every emulated instruction:
 - Unbox** any NaN(s) and cast back to double-precision (loss-less).
 - Operate** directly on double-precision values.
 - Round** to the low-precision and box the result into a “teeny” NaN.



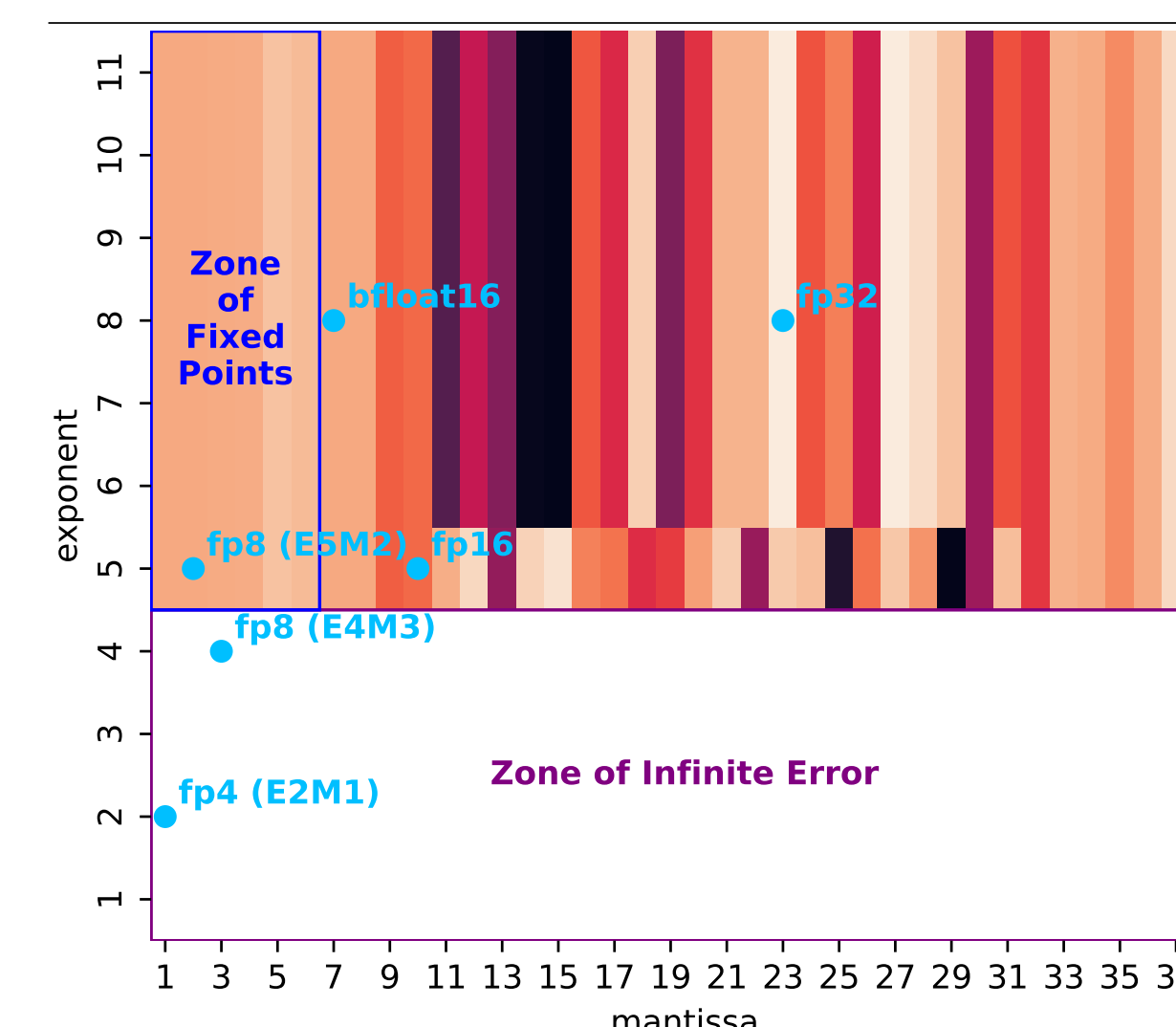
Evaluation/Performance

- Fire and forget** exploration of all exponent and mantissa sizes smaller than 52-bits.
- Automatically identifies which floating-point precisions the **unmodified** program can tolerate.

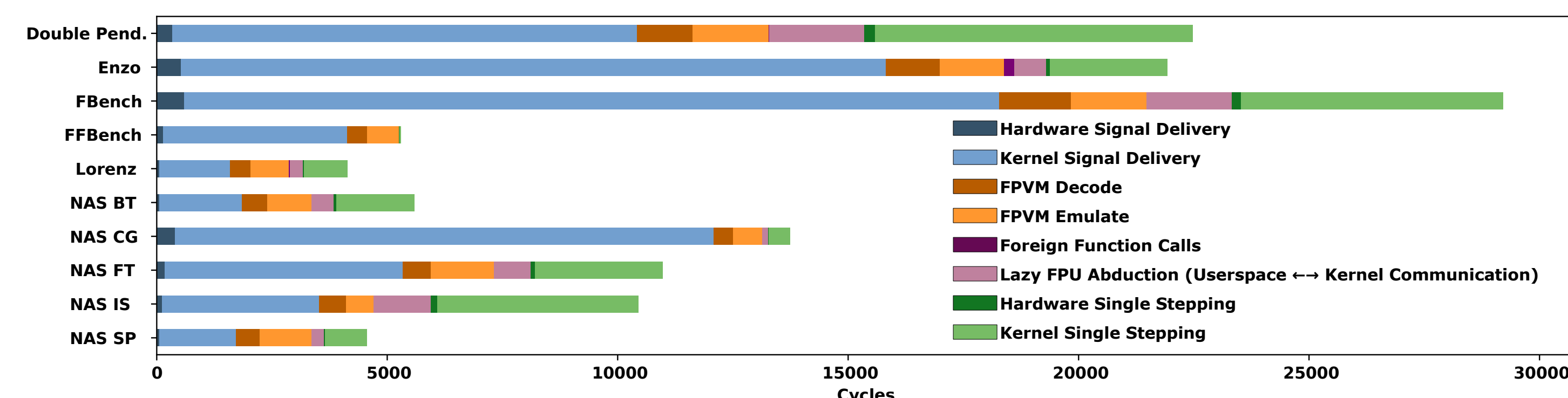
F Bench Error (Convergent)



Lorenz Attractor Error (Chaotic)



Amortized Cost Per Instruction

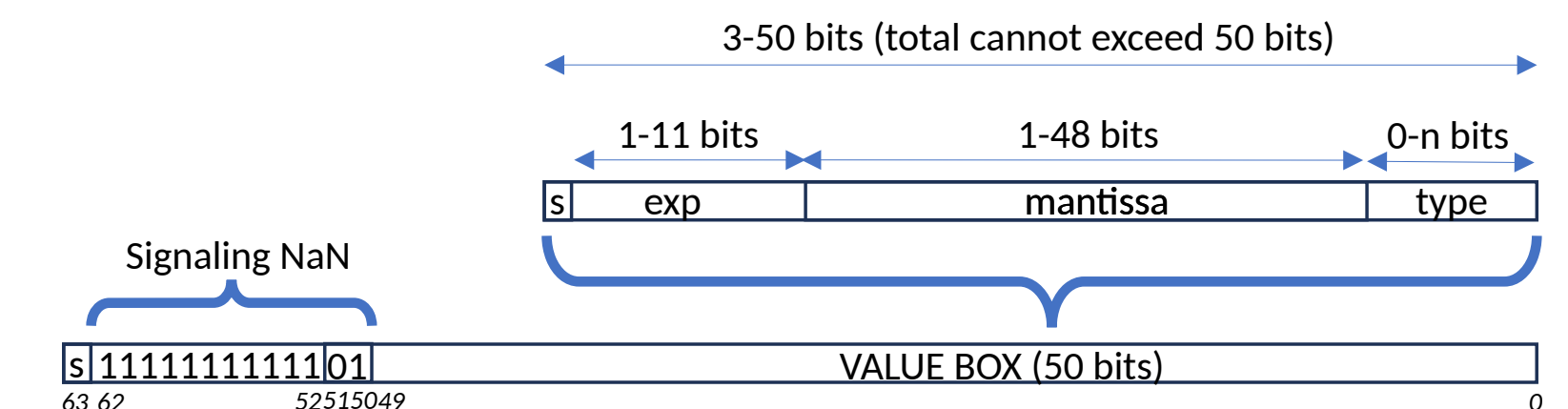


Largest overhead due to slow Linux kernel signal delivery.

Second largest due to shared integer-vector/floating-point register set.

Mixed Precision

- Most programs are not amenable to *uniformly* lowering the floating point precision.
- To address this “teeny” allows encoding precision within the value itself through an optional **type** field.



- The exact precision used is then set on a **value-by-value** or **instruction-by-instruction** as configured by the programmer.

Future Work

- The largest per-instruction overheads are caused by slow interrupt handling/signal delivery in the Linux kernel. Extending existing optimizations for FPVM such as trap short circuiting to work with Lazy FPU Abduction can potentially drastically reduce or eliminate these costs [3].
- Because “teeny” values are represented as NaN(s) we can disable Lazy FPU Abduction periodically to run at near-native speeds while minimally leaking higher precision into the result.

References

- Peter Dinda, Nick Wanninger, Jicheng Ma, Alex Bernat, Charles Bernat, Souradip Ghosh, Chris Kraemer, and Yehea Elmasry. FPVM: Towards a floating point virtual machine. In *Proceedings of the 31st ACM Symposium on High-performance Parallel and Distributed Computing (HPDC)*, June 2022.
- Kevin Hayes and Peter Dinda. Enabling floating point virtualization with tiny numbers. In *Proceedings of the 35th ACM Symposium on High-performance Parallel and Distributed Computing (HPDC)*, July 2026.
- Nick Wanninger, Nadharm Dhantravan, and Peter Dinda. Virtualization so light, it floats!: Accelerating floating point virtualization. In *Proceedings of the 34th ACM Symposium on High-performance Parallel and Distributed Computing (HPDC)*, July 2025.